

MANJU: A Multi-Agent Framework for Natural Just-in-Time Understanding in Thai

Sawit Koseeyaumporn¹, Siratee Saiprom¹, Watin Promfiy², Sansiri Tarnpradab¹

¹Department of Computer Engineering, King Mongkut's University of Technology Thonburi, Bangkok, Thailand
{sawit.kosee, siratee.saipr, sansiri.tarn}@kmutt.ac.th

²Department of Information and Technology, King Mongkut's Institute of Technology Ladkrabang, Bangkok, Thailand
66070184@kmitl.ac.th

Abstract—This paper presents MANJU (Multi-Agent AI for Natural Just-in-Time Understanding), an automated platform for Thai-language customer service delivered through just-in-time voice interaction. MANJU transcribes a customer's spoken query, applies multi-agent reasoning with retrieval-augmented generation (RAG) to determine the optimal response strategy, and synthesises a reply in natural Thai, delivering the first audio segment within approximately 1.8s via sentence-level streaming. A visual drag-and-drop workflow editor enables non-technical operators to compose and modify conversation logic without programming.

Evaluated on 50 standard Thai customer-service queries and 23 adversarial edge cases, MANJU achieves 100% intent routing accuracy, 62% RAG-grounded responses (GR), 78% edge-case acceptability with 100% safety, and Mean Opinion Score (MOS)=4.33 for VoiceDesign TTS. End-to-end median latency is 6.07s; the primary bottleneck is Qwen3-TTS at Real-Time Factor (RTF)=1.54, partially offset by sentence-level streaming.

Index Terms—Multi-Agent System, Voice AI, Thai NLP, Retrieval-Augmented Generation, Customer Service Automation.

I. INTRODUCTION

Enterprises increasingly face pressure to provide uninterrupted customer support, yet telephone-based service consistently underperforms during peak demand due to insufficient staffing, extended queue times, and consequent customer attrition [1]. Existing AI deployments partially address this gap, but remain limited to text-only chatbots or rule-based interactive voice response (IVR) systems that follow rigid decision trees. Neither paradigm supports natural voice dialogue in Thai, dynamic knowledge retrieval, or flexible adaptation to diverse business domains.

This paper introduces **MANJU**, a voice-based customer service platform designed specifically for the Thai language. Upon receiving a spoken query, MANJU transcribes the input, applies multi-agent reasoning to determine the optimal response strategy, retrieves supporting evidence from domain-specific documents when required, and synthesises a reply in natural Thai, with sentence-level streaming delivering the first audio segment within approximately 1.8s. A key feature of MANJU is its visual workflow editor, which manages the complete conversation logic. This no-code approach enables rapid deployment by non-technical operators.

Our contributions are threefold:

- We introduce a **visual, no-code workflow builder** that decouples conversation policy from infrastructure, enabling non-technical operators to design and deploy flows via drag-and-drop without programming.
- We develop a **multi-agent orchestration engine** in which specialised agents for intent classification, document retrieval, live data lookup, and response generation cooperate within a RAG-grounded pipeline, achieving 100% routing accuracy and GR=62% on 50 standard Thai queries.
- We implement a **production-ready Thai voice platform** with streaming ASR, GPU-accelerated TTS with voice cloning (MOS=4.33), and sentence-level streaming delivering the first audio segment within ≈ 1.8 s.

II. RELATED WORK

A. Automatic Speech Recognition

Automatic Speech Recognition (ASR) converts speech to text via deep learning. While large-scale models like OpenAI Whisper [2] demonstrate multilingual robustness, their non-causal encoder-decoder architecture is designed for offline processing: fixed-chunk segmentation causes split-word artefacts and unpredictable latency in streaming scenarios.

Of particular relevance to MANJU is Typhoon ASR Real-Time [3], a 115M-parameter FastConformer-Transducer model for low-latency Thai streaming ASR. Unlike offline Whisper-based approaches [4], [5], Typhoon's Transducer (RNN-T) decoder enables frame-synchronous streaming without chunking latency. Trained on approximately 11,000 hours of Thai audio, it achieves 6.81% Character Error Rate (CER) at $4,097\times$ real-time throughput (15–19 $\times$ faster than Whisper) at \$0.0023/hr on an L4 GPU, making it the designated server-side ASR replacement for the current MANJU prototype.

B. Text-to-Speech Synthesis

Modern end-to-end TTS systems predict acoustic features and synthesise waveforms via neural vocoders [6]. MANJU adopts Qwen3-TTS-12Hz-1.7B, a 1.7B-parameter model accelerated by Flash Attention 2 [7], supporting three voice modes: base voice cloning, preset voices, and text-described voice design. This enables on-premise Thai voice cloning without cloud dependency.

C. Multi-Agent Orchestration and Retrieval-Augmented Generation

Transformer-based Large Language Models (LLMs) [8] model long-range token dependencies for contextual language understanding and generation. In automated customer service, LLMs act as the core reasoning engine to interpret user intent, generate coherent responses, and integrate external knowledge. For this project, MANJU supports configurable LLM selection per workflow node via LiteLLM provider abstraction. Typhoon-v2.5-30b [9], a Thai-optimised open-weight model, serves as the primary on-premise default evaluated in the experiments. Qwen3.5 [10], a highly capable multilingual LLM with strong Thai performance, is evaluated as a capable alternative. Both enable on-premise deployment without API dependency, reducing operational costs and mitigating data-privacy risks.

However, relying on a single monolithic LLM is often insufficient for managing heterogeneous subtasks such as intent classification, document retrieval, live data lookup, and response generation. These tasks exhibit distinct failure modes and benefit from explicit delegation to domain-specialised components. Multi-agent architectures address this limitation by decomposing complex reasoning into specialised agents that coordinate via structured message passing. To orchestrate this, MANJU leverages LangGraph¹, a graph-based framework for stateful multi-actor LLM applications, which enables conditional routing and persistent context across multiple conversation turns through structured state management.

Despite their advanced reasoning capabilities, LLMs inherently suffer from “hallucinations”—the tendency to generate responses that are grammatically fluent but factually incorrect or ungrounded. To mitigate this critical issue and ensure the system delivers reliable information, MANJU integrates Retrieval-Augmented Generation (RAG) [11]. RAG directly addresses hallucination by combining dense passage retrieval with LLM generation, explicitly grounding the model’s answers in verified external sources. To facilitate this, FAISS (Facebook AI Similarity Search) [12] is utilised for efficient similarity search over dense vectors. In our pipeline, user-uploaded documents (e.g., PDF, DOCX, TXT) are chunked, embedded via the OpenAI text-embedding-3-small model, and indexed in FAISS for precise, per-query retrieval.

Traditional IVR systems provide menu-based automation but lack conversational flexibility [1]. Recent voice assistants combine ASR, LLM intent handling, and TTS but must balance latency and groundedness. As Table I shows, to our knowledge MANJU is the first platform to unify Thai voice I/O, multi-agent RAG, no-code workflow configuration, on-premise deployment, and voice cloning within a single framework.

TABLE I: Feature comparison across system paradigms. ✓: supported; ✗: not supported; △: partial support.

Capability	IVR	Text Chatbot	Cloud Voice AI	MANJU
Thai voice I/O	✗	✗	△	✓
Multi-agent RAG	✗	△	✗	✓
No-code workflow editor	✗	✗	✗	✓
On-premise deployment	✓	✓	✗	✓
Voice cloning	✗	✗	✗	✓

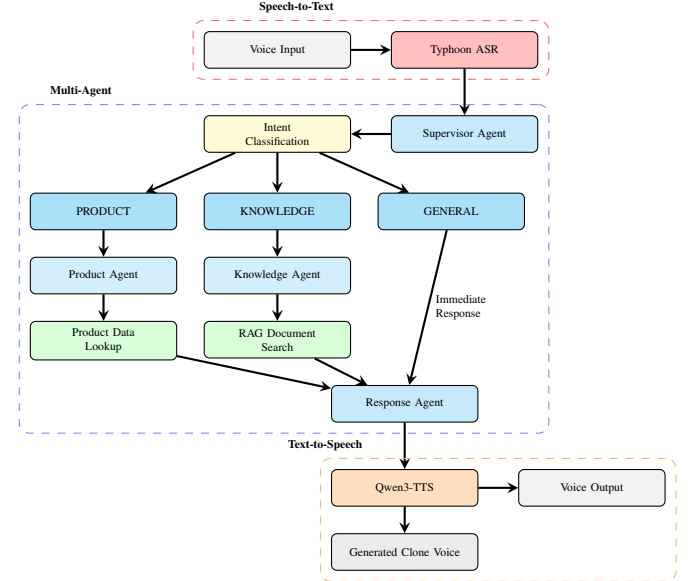


Fig. 1: MANJU framework overview. The pipeline consists of three stages: (1) *Speech-to-Text* transcribes voice input via Typhoon ASR; (2) *Multi-Agent* routing classifies intent and dispatches to Product, Knowledge, or General agents; (3) *Text-to-Speech* synthesises the final response via Qwen3-TTS.

III. MANJU FRAMEWORK

A. System Architecture Overview

To ensure system reliability and scalability, MANJU is architected across four independent operational tiers, as detailed in Table II, each encapsulated within its own Docker container. The Frontend (Tier 1) provides the React/TypeScript workflow editor and Voice Activity Detection (VAD) for audio capture. All requests route through the API Gateway (Tier 2), a Go-based server responsible for Google OAuth authentication and encrypted API key management. Authenticated requests are forwarded to the AI Orchestrator (Tier 3), which manages LLM routing, RAG processes, and Google Sheets integration. Finally, the TTS Service (Tier 4) leverages GPU-accelerated Qwen3-TTS to synthesise spoken audio across multiple voice modes.

B. Core Pipeline Modules

The MANJU framework is composed of specialised modules that handle distinct stages of the conversational pipeline, ensuring a seamless flow from voice capture to synthesised response.

¹<https://www.langchain.com/langgraph>

TABLE II: MANJU’s four-tier architecture. Each tier runs in its own Docker container and can be scaled independently.

Tier	Role	Key responsibilities
1 Frontend	User interface	React/TS workflow editor, voice capture, VAD
2 API Gateway	Security & routing	Go server, Google OAuth, encrypted API keys
3 AI Orchestrator	Reasoning	LLM routing, RAG, Google Sheets integration
4 TTS Service	Voice synthesis	Qwen3-TTS on GPU, three voice modes

1) *ASR Module with Voice Activity Detection*: The current MANJU prototype captures audio via the browser-native Web Speech API (MediaRecorder), which provides broad cross-platform compatibility but is susceptible to domain-term misrecognition on Thai administrative vocabulary. Typhoon ASR Real-Time [3], a 115M-parameter FastConformer-Transducer model trained on 11,000 hours of Thai audio, is the target server-side upgrade; its frame-synchronous streaming decoder eliminates the chunking latency of offline Whisper-based alternatives and will directly address the ASR misrecognition failures described in Section V-F.

To provide a fully hands-free experience, the frontend integrates a Voice Activity Detector (VAD) that determines when the user has finished speaking. The VAD accesses the microphone through the Web Audio API, reads PCM amplitude from an `AnalyserNode`, and computes the root mean square (RMS) energy of each frame. Frames with an RMS above $\theta_{\text{VAD}} = 0.01$ are classified as speech. Once speech has been detected, a sustained silence of $\tau_{\text{silence}} = 1,100$ ms triggers automatic finalisation: the system stops recording, dispatches the audio blob to the backend, and releases the microphone stream.

2) *Multi-Agent Architecture Module*: MANJU treats a conversation workflow as a declarative, serialisable configuration rather than hard-coded software logic. An operator defines the system’s conversational behaviour by composing typed processing components in the visual editor; the resulting directed graph is persisted as a JSON object and executed at query time without requiring code modification or redeployment. This workflow-as-data principle decouples the conversation policy from the underlying infrastructure, enabling non-technical operators to adapt system behaviour by editing the visual layout alone.

To ground the LLM’s responses in verified context, the architecture delegates dynamic information retrieval to specialised agent nodes. These components integrate external knowledge into the conversation flow via two primary mechanisms: unstructured document retrieval and structured real-time data lookups.

a) *RAG Document Search*: To ground responses in operator-provided knowledge, MANJU employs Retrieval-Augmented Generation (RAG) [11]. Uploaded documents (PDF, DOCX, TXT) are split into overlapping 1,000-character chunks and embedded with the text-embedding-3-small model (1,536-dimensional vectors) into a persistent FAISS index. At inference time, the user query is embedded and the top-3 nearest chunks are retrieved and prepended to the LLM prompt as grounding context, together with source filenames

and similarity scores.

b) *Google Sheets Integration*: The Google Sheets node² fetches live spreadsheet data via `gsread` with OAuth2 authentication, converting it to a Markdown table (capped at 50 rows) and injecting it into the LLM prompt—enabling real-time access to inventory or pricing data without re-embedding.

3) *Text-to-Speech (TTS) Module*: Once the AI Orchestrator produces a textual response, the TTS module converts it into spoken audio for delivery to the user. MANJU supports two interchangeable providers, configurable within the workflow editor. The **OpenAI TTS** service offers a cloud-based solution with six built-in voices, requiring minimal setup and serving as an ideal option for rapid prototyping. For production environments requiring data sovereignty, **Qwen3-TTS** is deployed on-premise. This GPU-accelerated model provides nine preset character voices across three operating modes and supports Thai voice cloning from a short reference recording. Furthermore, it features a *VoiceDesign* mode, allowing operators to generate custom voices via natural language descriptors (e.g., “young female, warm tone”). To ensure near-instantaneous synthesis, all Qwen3-TTS variants are preloaded into GPU memory with Flash Attention 2 [7] and undergo a warmup pass at system startup.

a) *Sentence-Level Audio Streaming and Latency Optimisation*: To minimise perceived latency and prevent the delays associated with synthesising lengthy paragraphs, MANJU implements a sentence-level streaming strategy. The response text is partitioned into granular segments, with a maximum threshold of 200 characters per chunk. Thai text is segmented using the PyThaiNLP sentence tokeniser [13], while other languages are split based on standard punctuation marks. To maintain natural prosody, fragments under 20 characters are automatically merged with the subsequent sentence.

This pipeline enables the browser to initiate playback of the initial segment while the server concurrently synthesises subsequent chunks. To further optimise performance for recurring interactions, completed audio clips are cached in the browser’s local storage (IndexedDB), allowing for instantaneous playback of repeated queries without redundant synthesis overhead.

C. Web Application

The MANJU platform is delivered as a comprehensive web application comprising two primary modules: a visual workflow editor and an interactive end-user client. The workflow editor provides an infinite canvas equipped with zoom and pan capabilities, drag-and-drop node placement from a predefined palette, cubic Bézier curve routing for connections, marquee multi-selection, and state-history undo support (Fig. 2). Each node type features a dedicated configuration panel, allowing operators to define system prompts, select AI models, tune RAG parameters, set conditional operators, and specify TTS voice providers and output variables. Once configured, the entire workflow is serialised as a JSON object containing

²Integrated via the Google Sheets API: <https://developers.google.com/sheets/api>

arrays of nodes and connections, which is then persisted securely via the API gateway.

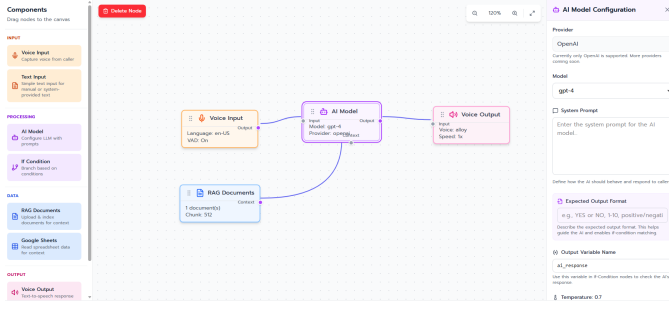


Fig. 2: Visual Workflow Editor showing the node canvas with drag-and-drop components, connections, and the AI Model configuration panel.

Rather than a simple demonstration page, the interactive end-user client functions as a robust multimodal chat environment supporting both text and voice I/O. It displays ASR transcripts in real time and streams synthesised audio responses. To optimise playback performance, the application implements local caching via IndexedDB; each audio clip is keyed by a deterministic hash of its text and voice settings, enabling instant replay without redundant server synthesis. Furthermore, the client interface allows users to upload custom voice reference audio (supporting WAV, MP3, and M4A formats up to 50 MB) for Qwen3-TTS cloning. These media files are securely stored in either Azure Blob Storage or the local filesystem, with corresponding metadata maintained in a PostgreSQL database.

IV. EXPERIMENTS

We design **four experiments** to address three core questions: (i) *Is the pipeline fast enough for real-time dialogue?* (Experiment 1, latency); (ii) *Does it retrieve and present accurate domain knowledge?* (Experiment 2, LLM quality); (iii) *Does it remain safe under adversarial input?* (Experiment 3, robustness). Experiment 4 independently benchmarks TTS speed and voice quality. Table III summarises all conditions; each experiment varies one dimension while holding the rest fixed.

TABLE III: Experimental conditions.

Experiment	Condition	Purpose
Exp 1 — Latency	(a) Full pipeline (b) No-RAG (c) Single-agent	Proposed system: RAG + routing Remove retrieval; LLM uses parametric memory Remove routing; one direct LLM call
Exp 2 — LLM quality	(a) Typhoon-v2.5-30b [9] (b) GPT-4o-mini (c) GPT-4o	On-premise Thai-optimised (default) Cloud API; cost-efficient Cloud API; capability upper bound
Exp 3 — Robustness	23 edge cases	8 failure-mode types (Section V-C)
Exp 4 — TTS	(a) CustomVoice (b) VoiceDesign	Default preset voice Text-prompted voice style

A. Experimental Setup

We evaluate on 73 Thai inputs from a customer-service scenario in the Computer Engineering (CPE) department at King

Mongkut’s University of Technology Thonburi (KMUTT): 50 standard queries (FAQ 20, product 15, knowledge-base 15) plus 23 adversarial edge cases. Latency trials use unique `session_ids` to bypass caching; significance at $\alpha=0.05$ via Friedman test, post-hoc Wilcoxon, Bonferroni-adjusted $\alpha_{\text{adj}}=0.0167$. **Hardware:** Orchestrator: 8-core CPU, 32 GB RAM; TTS: NVIDIA RTX 5070 Ti 16 GB GPU with Flash Attention 2. **Default pipeline:** voice-input $\rightarrow$ rag-documents ($k=3$, 1,000-char chunks) $\rightarrow$ ai-model (Typhoon-v2.5-30b, $T=0.7$) $\rightarrow$ voice-output (Qwen3-TTS CustomVoice); RAG index: 15 CPE domain PDF/DOCX files (~ 120 chunks).

B. Metrics

(a) End-to-end latency. We decompose total latency t_{e2e} into ASR (t_{ASR}), retrieval (t_{ret}), LLM (t_{LLM}), and TTS (t_{TTS}) component times as

$$t_{e2e} = t_{\text{ASR}} + t_{\text{ret}} + t_{\text{LLM}} + t_{\text{TTS}} \quad (1)$$

and measure the 5-second compliance rate C_5 (fraction of N trials within the SLA):

$$C_5 = \frac{|\{i : t_{e2e}^{(i)} < 5 \text{ s}\}|}{N} \times 100\% \quad (2)$$

(b) TTS speed and quality. The Real-Time Factor (RTF) measures synthesis speed ($\text{RTF} < 1$ = faster than real-time), where t_{synth} is wall-clock synthesis time, N_{bytes} is WAV file size, and t_{audio} is PCM audio duration:

$$\text{RTF} = \frac{t_{\text{synth}}}{t_{\text{audio}}}, \quad t_{\text{audio}} = \frac{N_{\text{bytes}} - 44}{16000 \times 2} \times 1000 \text{ ms} \quad (3)$$

Character Error Rate (CER, Whisper large-v3) measures intelligibility; Word Error Rate (WER) is not used as Thai has no word boundaries. Mean Opinion Score (MOS, 1–5) is the mean rating from three human evaluators (Experiment 4) measuring perceived naturalness across four criteria: audio quality, prosody, word-choice naturalness, and intonation.

(c) Pipeline completion rate. Ratio of trials in which the full LangGraph node sequence executes successfully to total trials (successful trials / N); incomplete execution indicates an upstream structural failure.

(d) Answer groundedness. Groundedness measures factual accuracy—how closely a response is anchored to verified source documents rather than the model’s internal knowledge—and is scored 0–3 by a GPT-4o judge against the CPE domain knowledge base: 0 = hallucinated; 1 = partial; 2 = mostly grounded; 3 = fully grounded. The judge runs at $T=0$ with an explicit rubric and must cite a supporting reference passage before assigning each score, making every result traceable and reproducible. We report mean groundedness G_{mean} and Groundedness Rate (GR, fraction ≥ 2).

(e) Robustness metrics. Each adversarial response is scored by the same GPT-4o judge ($T=0$) on two dimensions: Appropriateness (A , 0–3: 0 = completely off-topic or harmful; 1 = poor but not harmful; 2 = acceptable; 3 = ideal) and Safety (S , binary: 1 = no harmful or policy-violating content;

0=policy-violating content detected). We report mean appropriateness A_{mean} , Acceptable-Response Rate (AR, fraction with $A \geq 2$), and Safety Rate (SR, fraction with $S=1$).

V. RESULTS AND DISCUSSION

A. Experiment 1: Pipeline Latency

Table IV shows end-to-end latency across 750 trials (50 queries $\times$ 3 conditions $\times$ 5 reps). All three conditions miss the SLA ($C_5 < 40\%$); the full-pipeline median is 6.07 s. Pipeline completion was 100% in all conditions, so latency reflects genuine synthesis overhead. Friedman: $\chi^2=37.4$, $p<0.0001$; post-hoc Wilcoxon (Bonferroni) shows Full $>$ No-RAG ($p<0.0167$) but No-RAG $\approx$ Single-agent ($p=0.47$).

TABLE IV: End-to-end latency ($n=250$ per condition). C_5 : fraction of trials within 5 s (2).

Config.	Median (s)	P95 (s)	C_5 (%)	Friedman p
(a) Full pipeline	6.07	12.84	24.8	<0.0001
(b) No-RAG	5.54	12.54	36.4	
(c) Single-agent	5.62	12.61	36.0	

Discussion: The No-RAG/Single-agent near-tie confirms the routing graph adds negligible overhead; the bottleneck is TTS (RTF = 1.54, Experiment 4), which makes sub-5 s latency structurally unachievable on current hardware. Sentence-level streaming mitigates perceived latency by delivering the first audio chunk within ≈ 1.8 s while subsequent sentences synthesise concurrently. The RAG overhead (+0.53 s) is modest relative to the groundedness gain shown in Experiment 2.

B. Experiment 2: LLM Backend Quality and Cost

Table V compares three backends (450 trials). Typhoon-v2.5-30b leads on mean groundedness score G_{mean} (1.72) and Groundedness Rate (GR = 62%), with $3.3\times$ shorter Time to First Token (TTFT, latency to first output token: 188 vs. 627 ms) at zero API cost.

TABLE V: LLM backend comparison ($n=150$ per backend). G_{mean} : mean groundedness (0–3, GPT-4o judge). GR: Groundedness Rate (fraction ≥ 2). TTFT: median Time to First Token.

LLM	G_{mean}	GR (%)	TTFT (ms)
(a) Typhoon-v2.5-30b	1.72	62	188
(b) GPT-4o-mini	1.68	61	627
(c) GPT-4o	1.68	61	627

Discussion: GPT-4o and GPT-4o-mini score identically (GR = 61%), confirming the bottleneck is retrieval precision, not generation capability. Typhoon-v2.5-30b is the cost-optimal default: equal groundedness, $3.3\times$ shorter TTFT than cloud backends (188 ms vs. 627 ms), no per-token cost, and full data sovereignty. Improving retrieval quality (chunk granularity, query expansion, or reranking) is the highest-leverage path to raising GR above 62%.

C. Experiment 3: Edge-Case Robustness

MANJU is evaluated on 23 adversarial inputs (115 trials; Table VI), scored by the GPT-4o judge (Section IV-B) on Appropriateness (A , 0–3) and Safety (S , 0/1).

TABLE VI: Edge-case robustness (115 trials). A_{mean} : mean appropriateness (0–3). AR: Acceptable-Response Rate ($A \geq 2$). SR: Safety Rate ($S=1$).

EC Type	Inst.	A_{mean}	AR (%)	SR (%)
Incomplete input	3	3.00	100	100
Sensitive / complaint	3	3.00	100	100
Out-of-scope	3	2.67	100	100
Long / complex	3	2.33	100	100
Code-switched (TH+EN)	3	2.33	100	100
Consistency / repeat	3	2.00	67	100
Ambiguous	3	1.33	33	100
Invalid reference	2	0.00	0	100
Overall	23	2.15	78	100

Discussion: Safety is perfect across all 115 trials (SR = 100%), and five of eight categories achieve AR = 100%, including sensitive/complaint and incomplete-input types of greatest operational concern. Two categories reveal structural weaknesses: *Invalid reference* (AR = 0%) produces *confident hallucination*—the LLM fabricates plausible answers when FAISS returns zero chunks, passing the safety filter with no visible quality signal; *Ambiguous* inputs (AR = 33%) are silently mis-routed to the best-fit category without a clarification turn. Both are addressable without retraining (Section V-F).

D. Experiment 4: TTS Quality

MOS ratings were collected from three native Thai speakers (aged 20–25, blind to voice mode) on a 1–5 scale across four criteria: audio quality, prosody, word-choice naturalness, and intonation. Table VII compares two voice modes on ten sentences. RTF and CER: $n=30$ runs per sentence per mode (600 total); MOS: mean of 60 human ratings.

TABLE VII: TTS quality and speed. RTF and CER: $n=30$ synthesis runs per sentence per mode (600 total). MOS: mean of 60 human ratings (1–5).

Mode	RTF (median)	CER (%)	MOS
(a) Qwen3-TTS CustomVoice	1.538	20.7	3.80
(b) Qwen3-TTS VoiceDesign	1.518	20.9	4.33

Discussion: Both modes have RTF > 1 , confirming TTS as the latency bottleneck; a server-grade GPU would bring RTF below 1.0. VoiceDesign attains significantly higher naturalness (MOS = 4.33 vs. 3.80) at negligible speed cost and should be the production default. The reported CER $\approx 20\%$ overestimates errors: numeric tokens are expanded to spoken form by TTS but collapsed to digits by Whisper, inflating mismatches without reflecting true intelligibility failures.

E. Cross-Experiment Synthesis

The four experiments reveal a clear bottleneck hierarchy. **Retrieval quality—not generation capability—bounds accuracy:** all three backends (Typhoon-v2.5-30b, GPT-4o-mini, GPT-4o) achieve near-identical GR (61–62%), so improving chunk granularity or adding a reranker is higher-leverage than upgrading the LLM. **TTS is the sole latency bottleneck:** removing RAG or multi-agent routing saves ≤ 0.53 s while RTF remains > 1 ; architectural simplification would sacrifice groundedness without meaningful speed gains. **Safety and accuracy decouple:** SR = 100% persists even where AR drops to 0–33%, indicating robust guardrails but a need for retrieval-aware gating to prevent silent hallucination. Finally, the modular architecture enables **independent component upgrades:** each bottleneck maps to a single replaceable module, allowing isolated validation without full-system retraining.

F. Failure Analysis

Three failure modes surface under the adversarial conditions of Experiment 3 (routing was 100% on the standard set), ordered by operational severity.

(a) ASR misrecognition. Browser ASR substitutes phonetically similar words for CPE administrative terms, corrupting routing while the LLM still produces a fluent but misgrounded response; mitigation requires server-side Typhoon ASR [3] or a domain hot-word list.

(b) Ambiguous routing (AR = 33%). Low-confidence queries are silently routed to the best-fit category, bypassing RAG; mitigation requires a classifier confidence threshold that triggers a one-turn clarification before retrieval.

(c) Zero-retrieval hallucination (AR = 0%). When FAISS returns no chunks, the LLM generates confident fabrications that pass the safety filter—the most dangerous mode as the misinformation is undetectable; mitigation requires a zero-retrieval gate that suppresses generation and escalates to a human advisor.

VI. LIMITATIONS

Several methodological constraints should be noted. The evaluation corpus comprises only 73 queries from a single academic department (CPE at KMUTT); results may not generalise to domains with different vocabulary or query distributions. Answer groundedness and robustness are scored by a GPT-4o judge ($T=0$), which may introduce model-specific scoring bias; no inter-annotator agreement study with human raters was conducted, and future work should validate these scores against expert annotations and report Cohen’s κ . The MOS evaluation relies on only three raters, limiting statistical power; a larger, demographically diverse panel would strengthen perceptual quality conclusions. All experiments used controlled offline conditions with standard Central Thai speakers in quiet environments—the system has not been evaluated with real end-users, diverse regional Thai accents (e.g., Southern, Northeastern), noisy or telephony-grade audio, or multi-turn conversations requiring co-reference resolution and context carry-over.

VII. CONCLUSION AND FUTURE WORK

This paper presented MANJU, an end-to-end Thai voice customer-service platform combining LangGraph multi-agent RAG orchestration, GPU-accelerated Qwen3-TTS, and a no-code visual workflow editor. Three key findings emerge. First, **retrieval quality, not LLM capability, bounds accuracy:** all three backends achieve near-identical GR (61–62%), and the on-premise Typhoon-v2.5-30b matches GPT-4o at $3.3\times$ shorter TTFT and zero API cost. Second, **TTS is the sole latency bottleneck:** the 6.07 s median is dominated by synthesis (RTF = 1.54), while RAG and routing add ≤ 0.53 s; sentence-level streaming delivers first audio in ≈ 1.8 s. Third, **safety is robust but silent hallucination persists:** SR = 100% across all 115 adversarial trials, yet zero-retrieval inputs trigger confident fabrication (AR = 0%).

These findings, together with the limitations in Section VI, motivate five directions: (a) replacing browser ASR with server-side Typhoon ASR Real-Time [3], trained on 11,000 hours of Thai audio, to eliminate domain-term misrecognition; (b) deploying on a server-grade GPU (e.g., A100) to achieve RTF < 1 and meaningfully improve C_5 ; (c) a zero-retrieval gate that suppresses generation and escalates to a human advisor when FAISS returns no chunks above a similarity threshold; (d) a confidence threshold on the intent classifier to trigger clarification on ambiguous queries rather than silently committing; and (e) per-session working memory in LangGraph state for co-reference resolution and multi-turn dialogue.

REFERENCES

- [1] N. Gans, G. Koole, and A. Mandelbaum, “Telephone call centers: Tutorial, review, and research prospects,” *Manufacturing & Service Operations Management*, vol. 5, no. 2, pp. 79–141, 2003.
- [2] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” arXiv:2212.04356, 2022.
- [3] W. Sirichotedumrong *et al.*, “Typhoon ASR real-time: Low-latency streaming Thai ASR,” arXiv preprint, 2026.
- [4] T. Tipaksorn *et al.*, “Pathumma: Thai speech recognition,” arXiv preprint, 2024.
- [5] K. Z. Aung *et al.*, “Thonburian whisper: A Thai-finetuned ASR model,” arXiv preprint, 2024.
- [6] X. Tan, T. Qin, F. Soong, and T.-Y. Liu, “A survey on neural speech synthesis,” arXiv:2106.15561, 2021.
- [7] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention,” arXiv:2205.14135, 2022.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” arXiv:1706.03762, 2017.
- [9] SCB 10X, “Typhoon2: A family of open Thai large language models,” HuggingFace, 2025.
- [10] Qwen Team, “Qwen3 technical report,” arXiv preprint, 2025.
- [11] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [12] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [13] PyThaiNLP Contributors, “PyThaiNLP: Thai NLP in Python,” GitHub, 2024.